

Travel Planner PHP Project

ملخص فهم المشروع والكود والتستات

الملف ده معمول عشان تفهم المشروع بسرعة قبل المناقشة أو التسليم: السيستم بيعمل إيه، ملفات PHP متقسمة إزاي، التستات بتثبت إيه، وأساسيات PHP المهمة زي GET و POST و Session و Prepared Statements.

Testing
PHPUnit

Backend
PHP + MySQL

Architecture
MVC بسيط

الفهرس

1. فكرة المشروع باختصار
2. مين المستخدمين وأدوارهم
3. workflow الأساسي داخل السيستم
4. تقسيمة المشروع: Controllers / Models / Views
5. شرح ملفات PHP المهمة
6. شرح كود التستات وهدف كل Test Class
7. أساسيات PHP المستخدمة في المشروع
8. أهم نقاط الأمان والفاليديشن
9. ملخص سريع للمناقشة

1. المشروع بيتكلم عن إيه؟

Travel Planner هو Web Application لإدارة الرحلات الجماعية. الفكرة إن المستخدم يقدر يشوف رحلات، يحجز رحلة، يدفع، يتابع الأنشطة اليومية، يضيف مصاريف، يرفع مستندات، ويستقبل Notifications. أما الـ Leader فيقدر ينشئ رحلات ويقسمها لأيام ومحطات، ويدير الأنشطة المقترحة. والـ Admin عنده Dashboard لإدارة المستخدمين والحجوزات والتقارير.

للقائد Leader

ينشئ Trip، يضيف Itinerary و Stops، ويمتلك صلاحية Approve / Reject للـ Activities الخاصة برحلته.

للمستخدم العادي Member

يشوف الرحلات، يحجز، يدفع، يرفع مستند، يتابع Activities، ويختار Going / Not Going.

قاعدة البيانات

تخزن users, roles, trips, bookings, payments, activities, expenses, documents, notifications.

للادمن Admin

يشوف Dashboard فيها users/bookings/statistics، ويقدر يعدل أو يحذف مستخدمين.

2. الأدوار والصلاحيات

Role	المعنى	الصلاحيات الأساسية
role_id = 1	Admin	إدارة المستخدمين، متابعة الحجوزات، رؤية إحصائيات السيستم.
role_id = 2	Leader	إنشاء وإدارة الرحلات التي يقودها، والموافقة أو الرفض للأنشطة.
role_id = 3	Member	حجز الرحلات، الدفع، متابعة الأنشطة، رفع المستندات، استلام الإشعارات.

أهم قاعدة في الصلاحيات: الواجهة ممكن تخفي زر، لكن الحماية الحقيقية لازم تكون في الـ Controller. يعني حتى لو المستخدم كتب URL بإيده، السيرفر لازم يرفض لو مش صاحب

الصلاحية.

3. ال Workflow الأساسي

1. المستخدم يعمل Sign in أو Register.
2. السيستم يحفظ بياناته في Session بعد تسجيل الدخول.
3. لو Admin يروح Admin Dashboard، ولو User يروح Homepage.
4. ال Leader ينشئ Trip ويضيف itinerary days وstops.
5. ال Member يشوف الرحلات ويحجز Trip.
6. بعد الحجز، المستخدم يقدر يدفع وتتحول حالة الحجز إلى confirmed.
7. المستخدمم يقترحوا Activities على itinerary معين.
8. ال Leader الرحلة يوافق أو يرفض activity.
9. Notifications بتوصل لصاحب العلاقة، زي activity proposed أو approved/rejected.

4. تقسيمة المشروع

Folder	وظيفته
Controllers/	المنطق الرئيسي: يستقبل Requests، يتأكد من البيانات، يكلم الموديل أو الداتابيز، ويرجع Redirect/Response.
Models/	Classes بتمثل الجداول أو العمليات القريبة من قاعدة البيانات، زي Activity وExpense.
Views/	صفحات HTML/PHP اللي المستخدم ببشوفها: login, register, trips, dashboard.
tests/	Unit tests باستخدام PHPUnit عشان نتأكد إن العمليات الأساسية شغالة.
sql/	ملفات تعديل أو إنشاء جداول مساعدة.

5. شرح ملفات PHP المهمة

Controllers

الملف	ببعمل إيه؟
DBController.php	مسؤول عن فتح وغلق اتصال MySQL باستخدام mysqli. معظم الملفات بتستخدمه قبل أي Query.
AuthController.php	تسجيل الدخول والتسجيل والخروج. ببستخدم password_hash و password_verify، وبيحفظ بيانات المستخدم في Session.
TripController.php	إنشاء الرحلة، إنشاء itinerary و stops، تحديث أو حذف trip مع التحقق من صلاحية owner/admin.
BookingController.php	إنشاء booking، منع الحجز المكرر، وإلغاء الحجز لصاحب الحجز فقط.
PaymentController.php	يتأكد إن المبلغ يساوي تكلفة الحجز، يسجل payment، ويحول booking إلى confirmed.
ActivityController.php	إنشاء activity، موافقة/رفض leader، وتسجيل RSVP going أو not_going.
ExpenseController.php	إضافة أو حذف مصاريف الرحلة وربطها بـ trip.
EmergencyController.php	إضافة emergency contacts للرحلة.
InviteMemberController.php	دعوة مستخدم للانضمام لرحلة وتسجيله في trip_member وإرسال notification.
DocumentController.php	رفع مستندات المستخدم مثل passport/ID وتخزين اسم الملف في جدول document.

Models

الملف	فكرته
activity.php	Class فيه create/getByItinerary/approve/reject للأنشطة.
expense.php	Class لإضافة وقراءة وحذف expense.
emergencyContact.php	Class لإضافة وقراءة emergency contacts المرتبطة بالرحلة.

Models بسيطة فيها properties بتعبر عن أعمدة الجداول.	booking.php / payment.php
تمثيل بيانات المستخدم والدور.	users.php / role.php
Repository بسيط لقراءة كل الرحلات.	TripRepository.php

Views

الملف	بيظهر للمستخدم إيه؟
Views/Auth/register.php	Form تسجيل حساب جديد، يرسل البيانات لـ AuthController.
Views/Auth/signin.php	Form تسجيل الدخول، وبعد النجاح يوجه Admin أو User حسب role_id.
Views/User/homepage.php	الصفحة الرئيسية للمستخدم وفيها روابط الرحلات والحجوزات والمستندات والإشعارات.
Views/User/viewTrips.php	عرض الرحلات المتاحة وتفاصيل itinerary و emergency contacts و زر booking.
Views/User/Trips/createTrip.php	Form إنشاء Trip جديد، خاص بالـ Leader/Admin.
Views/User/Trips/list.php	عرض الرحلات التي يملكها الـ Leader، والـ Admin يرى الكل.
Views/User/activityManagement.php	إدارة الأنشطة الخاصة بـ itinerary: propose activity و Approve/Reject للقائد فقط.
Views/User/viewBookings.php	عرض حجوزات المستخدم والدفع أو الإلغاء ودعوة أعضاء.
Views/Admin/adminDashboard.php	لوحة تحكم الأدمن: users, bookings, reports, edit/delete users.

6. شرح التستات

التستات مكتوبة بـ PHPUnit. كل Test Class يجهز بيانات في setUp، ينظفها في tearDown، وبعد ذلك كل function تبدأ بـ test... تختبر سيناريو معين باستخدام Assertions.

bootstrap.php

ده الملف اللي PHPUnit بيشتغله قبل التستات. بيعمل include للـ Models و Controllers المطلوبة، وبيوفر helper اسمها غالباً getConnectionTest() عشان كل test يفتح اتصال بقاعدة البيانات.

```
protected function setUp(): void
{
    $this->conn = getConnectionTest();
    // create test data
}

protected function tearDown(): void
{
    // delete test data
    $this->conn->close();
}
```

DBControllerTest.php

بيأكد إن الاتصال بقاعدة البيانات شغال، وإن اسم الداتايز صحيح، وإن الجداول الأساسية موجودة. ده مهم لأن باقي التستات كلها معتمدة على DB connection.

AuthControllerTest.php

Test	يثبت إيه؟
Register new user	إدخال user جديد في جدول users ينجح.
Duplicate email	الإيميل المتكرر يتم اكتشافه عن طريق SELECT.
Correct credentials	password_verify بيرجع true مع الباسورد الصحيح.

Wrong password	password_verify يرجع false مع باسورد غلط.
Non-existent email	الاستعلام يرجع 0 rows لو الإيميل مش موجود.
Password hashing	الباسورد المخزن لا يساوي النص الأصلي، لكن password_verify يشتغل.

TripControllerTest.php

بيختبر CRUD الخاص بالرحلات: إنشاء Trip، قراءتها بال ID، تحديث الاسم أو الميزانية، حذفها، قراءة Trip غير موجودة، التأكد من تخزين budget بشكل صحيح، والتأكد إن end_date بعد start_date.

BookingControllerTest.php

يبركز على booking flow: إنشاء booking، منع duplicate booking لنفس user ونفس trip، وتغيير حالة الحجز عند الإلغاء.

PaymentControllerTest.php

بيختبر الدفع: لو المبلغ صح يتسجل payment بحالة successful ويتحول booking إلى confirmed. ولو المبلغ مختلف عن تكلفة booking، التست يتأكد إن mismatch متشاف.

ActivityTest.php

بيختبر activity model: إنشاء activity بحالة pending، وبعدها approve يحول status إلى approved، و reject يحول status إلى rejected.

ExpenseTest.php

بيختبر إضافة expense مرتبطة برحلة، قراءتها، وحذفها من قاعدة البيانات.

آخر تشغيل للتستات على المشروع كان: **31 OK, 57 assertions, tests.**

7. أساسيات PHP المستخدمة في المشروع

PHP داخل HTML

PHP يقدر يكتب HTML ديناميكي. أي كود بين `<?php>` ... `<?>` بيتنفذ على السيرفر قبل ما الصفحة توصل للمتصفح.

```
<h1>Welcome, <?= htmlspecialchars($user['name']) ?></h1>
```

GET

GET بتجيب بيانات من الرابط. مثال: `edit.php?id=5` هنا نقدر نقرأ `id` بـ `$_GET['id']`.

```
$id = isset($_GET['id']) ? (int)$_GET['id'] : 0;
```

GET مناسب للقراءة أو التنقل، لكن مش أفضل اختيار للحذف أو العمليات الحساسة لأنه ظاهر في URL.

POST

POST بتستخدم غالبا مع forms لما نرسل بيانات زي `login/register/create trip`. البيانات بتكون في `$_POST`.

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $email = $_POST['email'];
    $password = $_POST['password'];
}
```

Session

Session بتخزن بيانات المستخدم بين الصفحات. بعد `login` بنحفظ `user id` و `role` في `$_SESSION`.

```
session_start();
$_SESSION['user'] = [
    "id" => $row['user_id'],
    "email" => $row['email'],
    "role_id" => $row['role_id']
];
```

require_once

يستخدم لاستدعاء ملف PHP مرة واحدة فقط. مثلا Controller يحتاج Model أو DBController.

```
require_once '../Controllers/DBController.php';
require_once '../Models/users.php';
```

Prepared Statements و mysqli

mysqli هي طريقة PHP للتعامل مع MySQL Prepared Statement بتحمي من SQL Injection لأنها تفصل كود SQL عن البيانات القادمة من المستخدم.

```
$stmt = $conn->prepare("SELECT * FROM users WHERE email = ? LIMIT 1");
$stmt->bind_param("s", $email);
$stmt->execute();
$result = $stmt->get_result();
```

bind_param types

الحرف	المعنى	مثال
s	string	email, name, status
i	integer	user_id, trip_id
d	double/float	budget, cost, amount
b	blob	ملفات binary نادرة الاستخدام هنا

password_verify و password_hash

ممنوع تخزين الباسورد password_hash plain text. password_verify بيحوله لقيمة آمنة، و password_verify بتأكد إن الباسورد اللي المستخدم كتبه يطابق الهاش.

```
$hash = password_hash($password, PASSWORD_DEFAULT);

if (password_verify($password, $row['password'])) {
    // login success
}
```

header Redirect

بعد عملية ناجحة بنقل المستخدم لصفحة ثانية باستخدام header. لازم بعدها نعمل exit عشان الكود اللي بعده مايكملش.

```
header("Location: ../Views/User/homepage.php");  
exit();
```

htmlspecialchars

لما نطبع بيانات جاية من المستخدم في HTML لازم نستخدم htmlspecialchars عشان نقلل خطر XSS.

```
<td><?= htmlspecialchars($row['name']) ?></td>
```

8. نقاط أمان وفاليديشن مهمة

- التحقق من login قبل أي صفحة حساسة باستخدام Session.
- التحقق من role_id قبل عمليات Admin أو Leader.
- التحقق من ownership: ال leader يعدل أو يحذف رحلاته فقط.
- منع الباسورد الأقل من 8 حروف من السيرفر وليس من الواجهة فقط.
- استخدام Prepared Statements بدل دمج قيم المستخدم داخل SQL مباشرة.
- استخدام htmlspecialchars عند عرض بيانات المستخدمين.
- عدم الاعتماد على إخفاء الزر فقط، لأن المستخدم ممكن يكتب URL بنفسه.

9. ملخص للمناقشة

لو اتسألت المشروع بيعمل إيه: قل إنه Travel Planner لإدارة الرحلات الجماعية، فيه roles مختلفة، وفيه flows للحجز والدفع والأنشطة والمصاريف والإشعارات. استخدمنا PHP مع MySQL و MVC بسيط، والتستات بـ PHPUnit بتغطي أهم العمليات زي .auth/trips/bookings/payments/activities/expenses

ولو اتسألت عن أهم جزء تقني: ركز على Session لل role_id، authentication، authorization، و Prepared Statements لحماية SQL، و password_hash لحماية الباسورد.

.Generated for Travel Planner project summary